



ResumeMind AI Project Report

Submitted To: Faisal Hafeez

Submitted By: M. Mudassar Nazar & Amjad waseem

Department: Computer Science

University: University OF Layyah

Course Title: Artificial Intelligent

Submission Date: 06/06/2026

Executive Summary

The AI Resume Parser is a web-based intelligent recruitment system developed to automate the process of resume screening and candidate evaluation. The increasing number of job applications received by organizations makes manual resume analysis time-consuming, labor-intensive, and prone to human error. This project addresses these challenges by utilizing Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to efficiently analyze resumes and assist recruiters in making informed hiring decisions.

The system is developed using React.js for the frontend and Node.js for the backend. It automatically extracts important information from resumes, including personal details, technical skills, educational qualifications, work experience, and certifications. The extracted information is then analyzed and compared with job descriptions to determine the suitability of candidates for specific positions.

The AI Resume Parser employs NLP for text processing and information extraction, while rule-based AI and text-matching algorithms are used to calculate candidate scores, rank applicants, and classify them into shortlisted, pending, or rejected categories. The system also provides AI-based recommendations for resume improvement and suggests suitable job roles based on the candidate's skills and qualifications.

A key feature of the system is its ability to process multiple resumes simultaneously, significantly reducing the workload of recruiters and improving the efficiency of the hiring process. Automated candidate ranking and job matching help organizations identify qualified applicants quickly while minimizing bias and manual effort.

The proposed system offers several benefits, including faster resume screening, improved accuracy in candidate evaluation, reduced recruitment costs, and intelligent decision support for HR professionals. Additionally, candidates benefit from personalized feedback and recommendations that can help improve their resumes and career opportunities.

Overall, the AI Resume Parser demonstrates the practical application of Artificial Intelligence in modern recruitment by integrating NLP, information extraction, text matching, recommendation systems, and rule-based AI techniques to create an efficient, accurate, and user-friendly hiring support platform.

📖 Table of Contents

1. Introduction

- Background
- Problem Statement

2. AI Capabilities & Features

3. How the AI Matching Engine Works

4. System Workflow

5. Project Directory Structure

6. Installation & Setup

- Prerequisites
- Backend Setup
- Frontend Setup

7. API Documentation

8. Dashboards & Usage

9. Technologies Used

10. Conclusion

1. Introduction

The AI Resume Parser is an intelligent web-based recruitment system designed to automate the resume screening and candidate evaluation process. Traditional recruitment methods require human resource (HR) professionals to manually review hundreds of resumes, which is time-consuming, error-prone, and inefficient. The proposed system utilizes Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to extract relevant information from resumes and compare it with job requirements.

The system automatically analyzes resumes, extracts important details such as personal information, skills, education, work experience, and certifications, and generates an evaluation report. It calculates a matching score by comparing the candidate's qualifications with the job description and provides AI-based recommendations for both recruiters and candidates. The system can process multiple resumes simultaneously and classify applicants into shortlisted, pending, or rejected categories based on predefined criteria.

The project is developed using React.js for the frontend and Node.js for the backend, providing an interactive and efficient platform for modern recruitment.

Background

Recruitment is one of the most important processes in any organization. Companies often receive a large number of applications for a single job position. Manual screening of resumes requires significant time and effort, and recruiters may unintentionally overlook qualified candidates due to human limitations.

With advancements in Artificial Intelligence, organizations have started adopting automated recruitment systems to improve efficiency and accuracy. Resume parsing and candidate matching have become important applications of AI and NLP in Human Resource Management.

Natural Language Processing enables computers to understand and process textual information contained in resumes. By extracting skills, educational qualifications, work experience, and other relevant details, AI systems can automatically compare candidate profiles with job descriptions and generate intelligent recommendations.

The proposed AI Resume Parser addresses these challenges by integrating NLP techniques, rule-based AI, text matching algorithms, and recommendation mechanisms into a single platform for automated candidate evaluation.

Problem Statement

Traditional recruitment processes face several challenges:

- Manual resume screening is time-consuming.
- HR professionals must review hundreds of resumes for a single vacancy.
- Human evaluation may introduce bias and inconsistency.
- Matching candidate skills with job requirements is difficult and error-prone.
- Ranking multiple candidates manually is inefficient.
- Candidates often do not receive feedback regarding weaknesses in their resumes.
- Organizations need a faster and more accurate method for identifying suitable applicants.

Therefore, there is a need for an intelligent system that can automatically analyze resumes, compare them with job requirements, rank candidates, and provide recommendations for better hiring decisions.

2. 🧠 AI Capabilities & Features

ResumeMind AI leverages advanced NLP pipeline processing to analyze resumes and match them against job requirements.

🌟 Core AI Features

ATS Resume Parsing: Extracts structured meta-information from PDF uploads, including email addresses, phone numbers, education levels, years of experience, and technical skill sets.

Weighted ATS Matching: Calculates a multi-dimensional match score between the candidate's resume and a specific job description.

Missing Skills Gap Analysis: Identifies critical skills and qualifications requested in the job description that are missing from the applicant's resume.

Actionable Improvement Suggestions: Generates targeted recommendations (e.g., adding metrics/KPIs, acquiring specific degrees/certifications, or integrating industry-specific keywords) to help candidates optimize their resumes.

Career Role Recommendation: Maps the candidate's skills profile to specialized tech roles, such as:

- Frontend Developer
- Backend Developer
- Machine Learning Engineer
- DevOps Engineer

AI Recommendation fitment Paragraph: Formulates automated, structured recruiter fitment feedback (e.g., *Highly Recommended*, *Consider for Interview*, *Not Recommended*) detailing why the applicant fits or lacks qualifications.

🔍 NLP Capabilities

Semantic Keyword Extraction: Uses lemmatization (spaCy NLP pipeline) to process nouns and proper nouns, separating syntax variations (e.g., "developed", "developing", "developer" all map to the base lemma "develop").

Custom Skill Pattern Matching: Leverages regex and vocabulary dictionary search across a large taxonomy of developer skills to categorize languages, tools, and frameworks.

Semantic Fallback Similarity: If a job description contains no explicit nouns or skills, the system automatically falls back to spaCy's vector-based document similarity index to evaluate context fit.

3. 🏗️ How the AI Matching Engine Works

The match score is calculated using a **weighted multi-criteria formula** to ensure a balanced evaluation:

$$\text{Overall Score} = (\text{Skills Score} \times 0.40) + (\text{Experience Score} \times 0.25) + (\text{Education Score} \times 0.15) + (\text{Keyword Score} \times 0.20)$$

❖ Skills Match (40% Weight)

Compares the set of skills extracted from the job description (S_{JD}) and the resume (S_{Resume}):

$$\text{Skills Score} = \frac{|S_{JD} \cap S_{Resume}|}{|S_{JD}|} \times 100$$

If no specific skills are found in the JD, it evaluates resume skill density up to a maximum of 100%.

❖ Experience Match (25% Weight)

Extracts years of experience from the resume and searches the JD for required experience (defaults to 2 years if none specified).

If $\text{Resume Years} \geq \text{JD Required Years}$, score is 100%

Otherwise, it calculates a ratio: $\frac{\text{Resume Years}}{\text{JD Required Years}} \times 100$ (with a baseline minimum of **45%** if any experience is present).

❖ Education Match (15% Weight)

Determines education levels of the applicant and compares it to the JD's minimum education criteria (PhD, Master's, Bachelor's, Associate's).

- Meeting or exceeding requirements yields 100%.
- Being one level below yields 75%.
- Anything lower yields 50%.

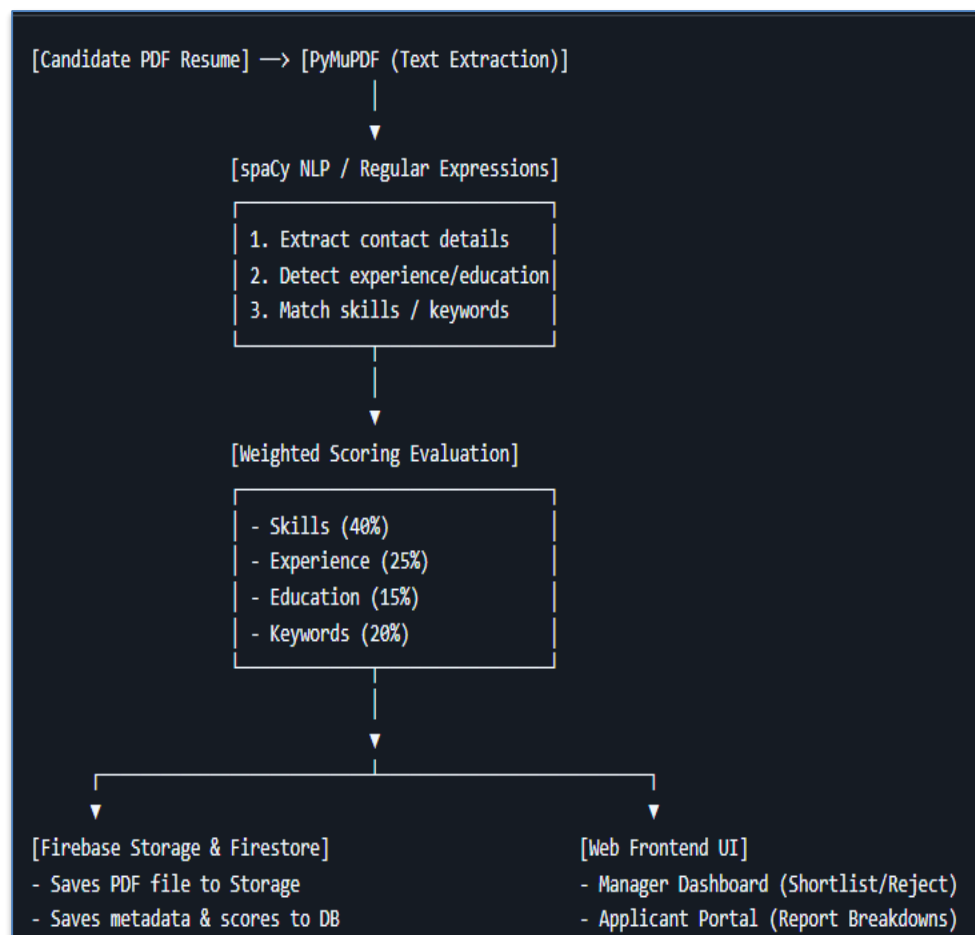
❖ **Keyword Match (20% Weight)**

Extracts nouns and proper nouns from the JD (K_{JD}) using spaCy and intersects them with all word lemmas from the resume (L_{Resume}):

$$\text{text}\{\text{Keyword Score}\} = \frac{|K_{JD} \cap L_{Resume}|}{|K_{JD}|} \times 100$$

4. 🔄 System Workflow

The end-to-end data pipeline is structured as follows:



5. ■ Project Directory Structure

⚙️ Installation & Setup

➤ Prerequisites

- Python 3.10+
- Node.js 18+ and npm
- Firebase Project setup

Backend Setup

1. Navigate to the backend directory:

```
bash
```

```
cd backend
```

2. Create and Activate a Virtual Environment:

Windows (PowerShell):

```
powershell
```

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

macOS/Linux:

```
bash
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

3. Install Dependencies:

```
bash
```

```
pip install -r requirements.txt
```

4. Install the spaCy NLP English Model:

```
bash
```

```
python -m spacy download  
en_core_web_sm
```

5. Set up Environment Variables:

Create a `.env` file in the `backend/` directory:

```
env
```

```
FIREBASE_SERVICE_ACCOUNT_PATH=./serviceAccountKey.json
```

```
FIREBASE_STORAGE_BUCKET=your-app-bucket-id.appspot.com
```

Make sure to download and place your Firebase `serviceAccountKey.json` credentials file inside the `backend/` directory.

6. Start the FastAPI Dev Server:

```
bash
```

```
uvicorn main:app --reload --port 8000
```

Frontend Setup

1. Navigate to the frontend directory:

```
bash
```

```
cd frontend
```

2. Install Packages:

```
bash
```

```
npm install
```

3. Set up Environment Variables:

Create a `.env.local` file in the `frontend/` directory:

```
env
```



```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

4. Start the Next.js Dev Server:

```
bash
```

```
npm run dev
```

5. Access the application:

Open
http://localhost:3000 in your web browser.

6. 📄 API Documentation

FastAPI automatically generates documentation at `http://localhost:8000/docs`. The core endpoints include:

- GET : Health check and welcome message.
- POST /api/upload: Extracts text from a PDF resume, parses its details, runs a baseline match, and stores the transaction metadata in Firestore.
- POST /api/match: Recomputes the weighted match score between an arbitrary resume and a specific job description.
- POST /api/storage/upload: Direct upload capability to Firebase Cloud Storage.
- GET /api/download/{filename}: Download parsed resumes stored on the backend.

7. 📊 Dashboards & Usage

📁 Manager Dashboard

Hiring managers can input or adjust the active job description in real-time, drag-and-drop multiple resume PDFs, and view an sorted leaderboard of candidates based on their AI Match score. From here, managers can view a comprehensive **AI report modal** containing:

- Scoring breakdowns
- Recommended career roles
- Improvement suggestions
- Recruiters can also update the candidate status (*Shortlist*, *Pending*, *Reject*).

👤 Applicant Portal

Designed as a self-service career tool, applicants can upload their resume to get immediate feedback. They receive an interactive **AI Report** showing where their resume ranks, what skills they are missing for typical engineering jobs, and precise suggestions on how to rewrite bullet points or add keywords to bypass ATS filters.

8. ⚙️ Technologies Used

Frontend: Next.js 15, React 19, TypeScript, Vanilla CSS (dynamic variables, dark/light theme support).

- Backend: FastAPI (Python), uvicorn.
- AI/NLP: spaCy (`en\_core\_web\_sm` English pipelines), PyMuPDF (PDF text mining).
- Database/Cloud: Firebase Admin SDK (Cloud Firestore & Cloud Storage).

9. Conclusion

The AI Resume Parser is an intelligent recruitment support system that combines Artificial Intelligence and Natural Language Processing to automate resume analysis and candidate evaluation. By extracting relevant information from resumes, matching candidate profiles with job descriptions, ranking applicants, and generating recommendations, the system simplifies the recruitment process for organizations while providing valuable feedback to job seekers. The integration of NLP, rule-based AI, and recommendation techniques makes the system an effective and practical application of Artificial Intelligence in modern hiring processes.